

Web access logs

Server side access logs: servers store subsets of request/response messages according to a predefined format (e.g., combined log format)

```
91.196.152.125 - - [15/Mar/2026:01:54:50 +0100] "GET / HTTP/1.1" 200 6995 "-" "Mozilla/5.0 (X11; Ubuntu; Linux x86_54; rv:134.0) Gecko/20100101 Firefox/134.0"
52.167.144.146 - - [15/Mar/2026:01:55:15 +0100] "GET /publications/PDF/Proxy.pdf HTTP/1.1" 200 180143 "-" "Mozilla/5.0 AppleWebKit/537.36 (KHTML, like Gecko; compatible; bingbot/2.0; +http://www.bing.com/bingbot.htm) Chrome/116.0.1938.76 Safari/537.36"
95.214.55.63 - - [15/Mar/2026:01:56:03 +0100] "GET / HTTP/1.1" 200 3666 "-" "-"
```

```
149.86.227.60 - - [15/Mar/2026:02:22:04 +0100] "GET / HTTP/1.1" 200 3666 "-" "Mozilla/5.0"
138.68.130.42 - - [15/Mar/2026:02:27:59 +0100] "SSTP_DUPLEX_POST /sra_{BA195980-CD49-458b-9E23-C84EE0ADCD75}/ HTTP/1.1" 400 3693 "-" "-"
45.205.1.8 - - [15/Mar/2026:02:33:31 +0100] "GET / HTTP/1.1" 200 6979 "-" "Mozilla/5.0"
176.65.149.235 - - [15/Mar/2026:02:35:41 +0100] "GET / HTTP/1.1" 200 3666 "-" "-"
43.153.47.201 - - [15/Mar/2026:02:47:19 +0100] "GET / HTTP/1.0" 400 535 "-" "-"
138.91.30.48 - - [15/Mar/2026:02:49:04 +0100] "GET /MCC/publications/workloads-clouds-preprint.pdf HTTP/1.1" 200 475390 "-" "Mozilla/5.0 AppleWebKit/537.36 (KHTML, like Gecko); compatible; ChatGPT-User/1.0; +https://openai.com/bot"
138.91.30.61 - - [15/Mar/2026:02:49:16 +0100] "GET / HTTP/1.1" 200 4884 "-" "Mozilla/5.0 AppleWebKit/537.36 (KHTML, like Gecko); compatible; ChatGPT-User/1.0; +https://openai.com/bot"
104.210.140.128 - - [15/Mar/2026:02:52:09 +0100] "GET /robots.txt HTTP/1.1" 404 3661 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36; compatible; OAI-SearchBot/1.0; +https://openai.com/searchbot"
```

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

50

Examples of fake User-Agent strings

```
198.167.198.239 - - [17/Apr/2023:23:29:22 +0200] "GET /wp-content/uploads/ HTTP/1.1" 404 295 "-" "Mozilla/5.0 (Linux; Android 7.0; SM-G892A Bulid/NRD90M; wv) AppleWebKit/537.36 (KHTML, like Gecko) Version/4.0 Chrome/60.0.3112.107 Moblie Safari/537.36"
```

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

51

Web error logs

Error logs are the first place to look in case of problems to a web server during testing and in operation (concept of *continuous monitoring*)

```
[Mon Mar 16 09:29:59.351405 2026] [autoindex:error] [pid 2337:tid 128241146787520] (13)Permission denied: [client 165.232.190.108:50393] AH01275: Can't open directory for index: /var/www/html/PEG/images/, referer: binance.com
[Mon Mar 16 18:14:55.569307 2026] [core:error] [pid 2338:tid 128242522502848] [client 47.250.153.97:35214] AH10244: invalid URI path (/cgi-bin/.%2e/.%2e/.%2e/.%2e/.%2e/.%2e/.%2e/.%2e/.%2e/bin/sh)
[Mon Mar 16 18:14:56.042266 2026] [core:error] [pid 2338:tid 128242402981568] [client 47.250.153.97:35216] AH10244: invalid URI path (/cgi-bin/%32%65%32%65/%32%65%32%65/%32%65%32%65/%32%65%32%65/%32%65%32%65/%32%65%32%65/bin/sh)
[Mon Mar 16 18:49:05.369358 2026] [core:error] [pid 2337:tid 128242136635072] [client 178.130.46.2:50448] AH10244: invalid URI path (/cgi-bin/.%2e/.%2e/.%2e/.%2e/.%2e/.%2e/.%2e/.%2e/.%2e/bin/sh)
```

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

52

Recap

Header fields are very powerful tools often used to add new features to the web (e.g., experimental security features)

Headers associated with HTTP request messages specify the settings of the user agents (e.g., browsers)

Headers associated with HTTP response messages specify how clients should handle the responses

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

53

Some header fields

Headers used to implement cookie mechanisms:

- **Set-Cookie:** used in HTTP response messages to set a cookie on the client device
- **Cookie:** used in HTTP request messages to send the web server a cookie

Header used to implement caching mechanisms:

- **Cache-Control:** used in both request and response messages to specify directives with respect to content cacheability

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

54

Another header field

Referer: used in request messages to store info about the immediate browsing history of a client (how the client “discovered” the content), that is, the URL of the web page from which the resource has been requested

It can be used for benign purposes, e.g., analytics, website design, caching optimization

There are also more problematic uses, such as tracking or stealing information, or even just side effects, such as inadvertently leaking sensitive information (e.g., disclose sensitive data in URL pathname or query parameters)

This might have undesirable consequences for user security and privacy

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

55

How to mitigate these security risks

Referrer-Policy header field tries to mitigate the security and privacy risks affecting the **Referer** header by specifying the referrer information to be included in the requests

Referrer-Policy: no-referrer

Referrer-Policy: same-origin

Referrer policies can also be implemented into the HTML code

```
<meta name="referrer" content="no-referrer">
```

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

56

Content Security Policy

Content Security Policy is an additional layer of security that helps to detect and mitigate security attacks (e.g., cross-site scripting, data injection) by restricting the objects that can be loaded and the URLs

This policy allows website developers to control the objects a page is permitted to fetch or execute and instruct the browser to place restrictions on the things that a code is allowed to do

Particularly useful for web applications that manage sensitive data

The policy can restrict the domains that browsers should consider as valid sources of executable scripts, thus preventing XSS attacks

The policy can also specify the allowed protocols, e.g., **https**, to mitigate packet sniffing attacks

It is implemented using the **Content-Security-Policy:response** header field set in the web server configuration or a **meta** tag in the HTML

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

57

Content Security Policy implementation

```
Content-Security-Policy: <policy-directive>;  
                        <policy-directive>
```

```
Content-Security-Policy: script-src https://abc.io/js
```

```
Content-Security-Policy: script-src maps.googleapis.com;  
                        img-src 'self'
```

```
Content-Security-Policy: upgrade-insecure-requests
```

```
<meta http-equiv="Content-Security-Policy" content="default-  
src 'self'">
```

The policy has a significant impact on how browsers will render pages

This requires a careful tuning and a very precise definition of the rules

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

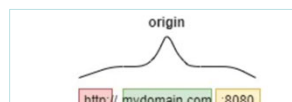
58

Same Origin Policy

Same Origin Policy (SOP) is a browser security feature that prevents dangerous interactions (e.g., gaining access to sensitive data) by restricting scripts on a web page from freely making cross-origin requests

It helps isolate potentially malicious documents, reducing possible attack vectors, e.g., it prevents a malicious website from running a JavaScript in a browser to read data from a page referring to another service

Origin is defined by three URL parameters: **scheme, hostname, port number**



A cross-origin request is a web request (either *fetch* or *XMLHttpRequest*) initiated from a script on one domain (e.g., **mydomain.com**) to another domain (e.g., **newdomain.com**).

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

59

Same Origin Policy implementation

The policy is implemented by browsers using the HTTP request **Origin**: header field used to specify the origin that caused the request (e.g., **Origin:https://www.abc.net**)

SOP blocks or denies reading cross-origin objects, i.e., objects retrieved from distinct origins are isolated from each other

SOP provides good protection against malicious scripts

Cross-origin requests should not be entirely prohibited

To decide whether to allow a cross-origin request, SOP is applied in combination with CORS policy

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

60

CORS policy

Cross-Origin Resource Sharing (CORS) extends and adds flexibility to the Same Origin Policy

CORS is a policy based on HTTP response headers that allows a web server to indicate origins from which a browser should be permitted loading objects

Web developers can control how their web applications react to cross-origin requests

Access-Control-Allow-Origin: <origin>

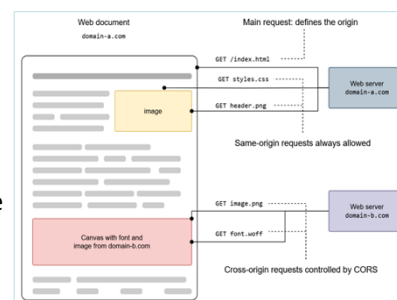
Access-Control-Allow-Methods: <GET, POST>

. . . .

CORS headers specify how a web application wants to respond to JavaScript requests coming from other web applications

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure



61

CORS implementation

A script at **api.example.com** attempts to make a cross-origin request to **frontend.example.com**

The browser detects a cross-origin request and initiates a CORS-preflight request

```
OPTIONS /api/resource HTTP/1.1
Host: api.example.com
Origin: https://frontend.example.com
Access-Control-Request-Method: POST
Access-Control-Request-Headers: Authorization, Content-Type
```

```
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: https://frontend.example.com
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Authorization, Content-Type
Access-Control-Allow-Credentials: true
```

Browser will grant access because origins, method and headers match

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

62

... and many more header fields

[Mozilla developer network](#) is an excellent reference for HTTP headers and for the HTTP in general

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

63

Performance & efficiency

Various factors affect the performance and efficiency of the web

- Design of web pages and web applications
- Web server hardware characteristics (e.g., computing power, RAM, storage), configuration settings (e.g., timeouts, max number of connections) and load intensity (i.e., number of concurrent requests)
- Network bandwidth, connectivity and traffic conditions

To improve QoS and user experience and avoid the “World Wide Wait”, it is necessary to minimize Page Load Time by controlling these factor

To reduce web server load and network traffic, HTTP requests should not be sent “too far” and HTTP responses should be “light”

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

64

Web caching

The objective of web caching is to store HTTP response messages close to the clients for possible reuse, thus avoiding accesses to the origin web servers

Benefits in terms of PLT of future equivalent requests, server load, network traffic and bandwidth usage

Content is cached as a result of a client access (pull model)

A cache consists of two components:

- local store of HTTP response messages
- management subsystem responsible of the storage, retrieval and removal of HTTP response messages

The implementation requires the definition of caches and of the associated header fields that control cache behavior or indicate cacheable response messages [[RFC 9111 “HTTP Caching”](#) published in June 2022]

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

65

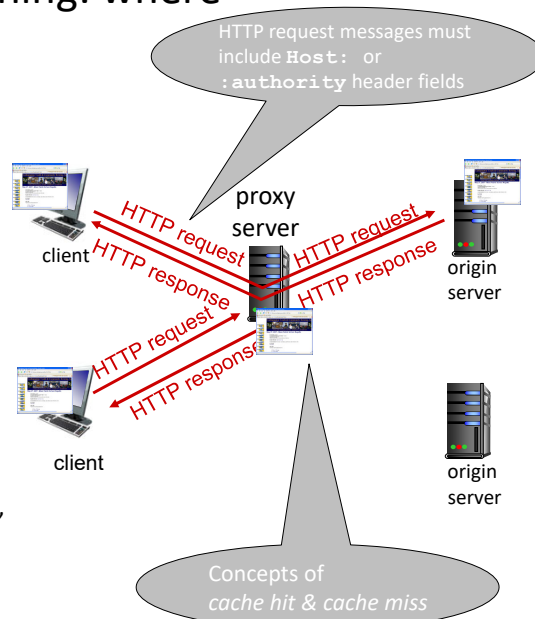
Caching: where

Browsers:

- Internal caches (in the RAM and on disk)
- Private accessible by the specific browser only

Proxy servers:

- Dedicated caching infrastructure
- All web accesses via proxy server
- Cache shared among all users of a proxy (e.g., ISP, company, organization)
- Homogeneous content

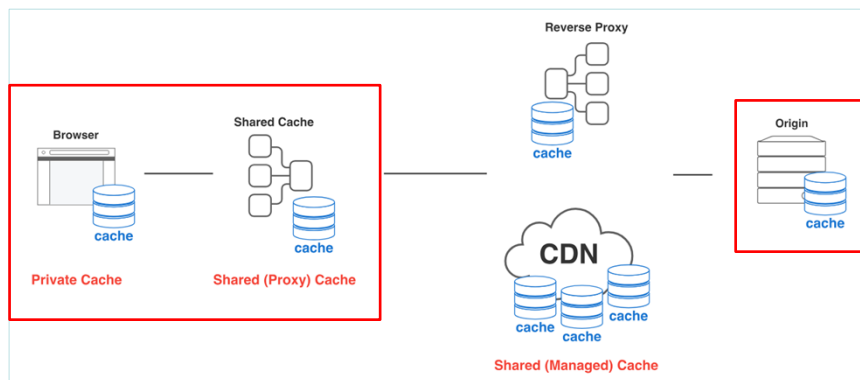


© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

66

Web caches



© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

67

Scenarios

A client could request a web page under three different scenarios:

- Direct access to the origin web servers and caching disabled
- Direct access to the origin web servers & browser cache enabled
- Access via a proxy server with caching enabled (& browser cache enabled)

The benefits of web caching depend on

- Hit/miss rates (e.g., fraction of “saved” transactions or of “saved” bytes)
- Cache location

Concept of “cache search”

- Availability
- Validity

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure